

From Similar Examples to Error Types: Extending PathFinder-PRM with Retrieval-Augmented Verification

Project Proposal Meeting · **Lei Tang**

Institute for Computational Linguistics

Heidelberg University

Presented by: **Pratik Goyal & Ashhad Raza**

Quadri

July 2026

“Have you seen the same problem in a slightly different form? Look at the unknown, and try to think of a familiar problem having the same or a similar unknown.” – George Pólya, How to Solve It (1945)



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

INSTITUT FÜR
COMPUTER-
LINGUISTIK

Where We Left Off: RetrievalPRM

Our seminar paper (RetrievalPRM, Zhu et al. 2025) built a step-level grader for math reasoning — a Process Reward Model, or PRM — that checks each step of a solution and marks it correct or wrong.

Its key idea:

- Before grading a step, first retrieve similar past questions and steps from a training pool
- Feed those examples into the grader's prompt as reference — an "open-book" exam instead of a closed-book one
- Two retrieval stages: question-level (fixes dataset shift) and step-level (fixes differences in reasoning style across models)

What it proved:

- A 7B RetrievalPRM beat 70B LLM critics (GPT-4o, Qwen2.5-72B) on ProcessBench, average F1 = 65.8
- Gains grew with difficulty — GSM8K +6.4 pts, up to OmniMATH +13.0 pts — evidence it fixes real distribution shift, not memorization

Same Problem, Two Different Fixes

Both papers build the same kind of grader, for the same job, tested on the same benchmark (ProcessBench). They just improve it in two different, unconnected ways.

RetrievalPRM (our seminar paper)

- Idea: look at similar past problems before grading
- Grading stays binary: correct / wrong
- Never tried separating error types

PathFinder-PRM (new paper)

- Idea: figure out the type of mistake before grading
- Grading becomes richer: error type + quality
- Never tried retrieval — grades every step "cold"

The gap between them is exactly our project: neither paper tried combining retrieval with error-typing.

Context: The Limits of a Single ✓ / ✗ Label

The problem with binary grading:

- A step can be "wrong" in very different ways — a calculation mistake, a contradiction with an earlier step, or just a messy, inefficient way to solve it
- A single correct / wrong label mixes all of these together and throws away useful information

Think of it like a doctor:

Instead of just saying "you're sick," a doctor first diagnoses what kind of illness it is — the diagnosis is what makes the treatment accurate. PathFinder-PRM applies this idea to grading math steps.

Why this matters for our project:

- It's a genuinely different grading structure from RetrievalPRM's binary judgment — a good target for testing whether retrieval's benefit transfers

Introducing the New Paper: PathFinder-PRM

Error Typing for Smarter Rewards: Improving Process Reward Models with Error-Aware Hierarchical Supervision

Pala, Sharma, Zadeh, Li, Poria (2025) — DeCLaRe Lab

One-line idea:

Before marking a step right or wrong, first work out what kind of mistake it is — a maths error, a consistency error, or (if there's no error) how good/efficient the step is.

Fully open and reproducible:

- Code: github.com/declare-lab/PathFinder-PRM
- Model: [declare-lab/PathFinder-PRM-7B](#) (Hugging Face)
- Data: [declare-lab/PathFinder-600K](#) (Hugging Face)

Core Idea: Hierarchical Error-Aware Grading

Stage 1 — Error Classification

- Math error? — is there an actual calculation or logic mistake in this step?
- Consistency error? — does this step contradict something said in an earlier step, even if its own maths looks fine?

Stage 2 — Optimality Estimation

- Only runs if Stage 1 found no error
- Scores how good/efficient the step is — is it a clean, direct step, or unnecessarily long-winded and roundabout?

So instead of one word ("correct" or "wrong"), the grader now returns a small report: error type (if any) + a quality score.

Why Split the Judgment?

Lumping everything into one "right or wrong" label hides useful information. Two very different steps can end up with the exact same label:

- A step that is flat-out wrong (a genuine calculation mistake)
- A step that is technically correct, but sloppy, inefficient, or badly written

Treating these the same makes the grader worse at catching subtle mistakes — and worse at explaining why a step was marked down.

Training data behind it:

- PRM800K (same human-labeled data our seminar paper used) + RLHFlow Mistral traces, relabeled with Math error / Consistency error / Optimality
- $\approx$ 400K–600K newly-labeled step examples, released publicly

Data & Training Recipe

Training Data

- PRM800K — human-annotated step labels
- RLHFlow Mistral traces — auto-annotated
- Relabeled into 3 dimensions: Math error / Consistency error / Optimality
- $\approx$ 400K–600K step-level examples total

Base Model & Fine-Tuning

- Qwen 7B family base model (same size class as RetrievalPRM's base)
- LoRA fine-tuning — lightweight, cheap, no full retraining
- Hierarchical objective: classify error type first, then optimality

Source: Pala, Sharma, Zadeh, Li, Poria (2025), arXiv:2505.19706

Headline Results: Best Open PRM on Two Benchmarks

On ProcessBench (same benchmark our seminar paper used):

- PathFinder-PRM-7B → average F1 = 69.5 — best among open-source PRMs, on every sub-dataset (GSM8K, MATH, OlympiadBench, OmniMATH)

On PRMBench (a harder benchmark for subtle error detection):

- Score = 67.7 — best among open PRMs, even beating some proprietary LLM judges

How this compares to our seminar paper:

- RetrievalPRM-7B scored 65.8 average F1 on ProcessBench — PathFinder-PRM's 69.5 is higher, using a different technique (error-typing, not retrieval)

Two independent ideas, two independent gains — neither has been tried together.

What's Already Built: declare-lab/PathFinder-PRM

The authors' GitHub repo runs end-to-end today — three ready-made pipelines:

Component	What it does
src/data_generation	Auto-relabels PRM800K + RLHFlow data with Math / Consistency / Optimality labels
src/training	Trains the hierarchical PRM — error classifier, then optimality estimator
src/evaluation	Tests on ProcessBench, PRMBench, and reward-guided generation; reproduces published numbers

Good and a problem: we have a solid working base, but re-running it as-is would just be reproduction, not our own project.

The Gap: What Neither Paper Tried

RetrievalPRM

- Uses retrieval to warm up the grader
- Grading output: binary (correct / wrong)
- Never classifies error type

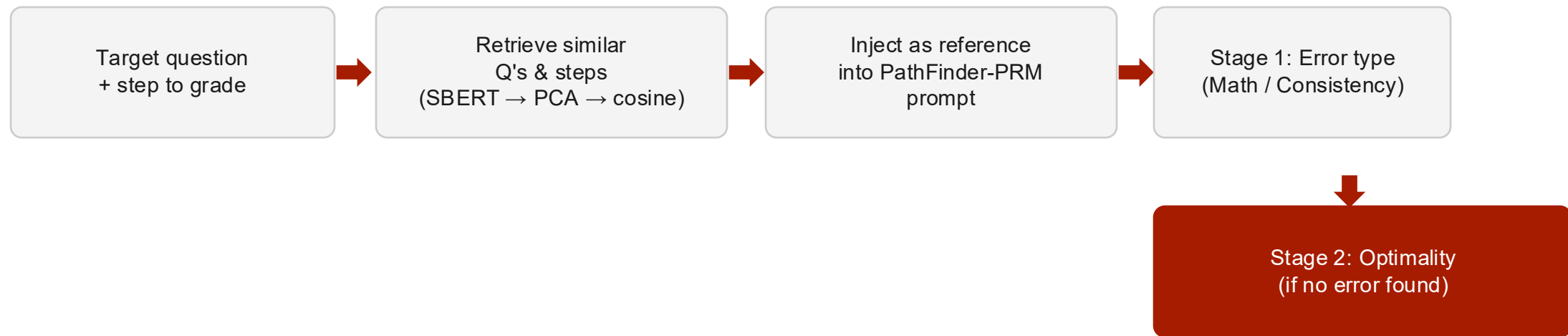
PathFinder-PRM

- Grades every step "cold" — no retrieval
- Grading output: error type + quality score
- Never uses reference examples

Our research question:

If we add retrieval into PathFinder-PRM's error-typing process, does it get better at identifying the correct error type — especially on the hardest, most out-of-distribution problems?

Our Proposal: Retrieval-Augmented PathFinder-PRM



The only new piece:

A retrieval step (borrowed exactly from RetrievalPRM) inserted before PathFinder-PRM's existing, unmodified error-classification stage. No retraining of PathFinder-PRM's weights.

Technical Plan: Four Steps

1

Build a retrieval system

Turn each question into a fingerprint with Sentence-BERT, shrink it with PCA, compare with cosine similarity — exactly RetrievalPRM's method.

2

Add examples into the prompt

Before PathFinder-PRM decides the error type, show it 1–2 similar solved examples as reference, like an open-book exam.

3

Run both versions & compare

Version A: PathFinder-PRM as-is (baseline). Version B: + our retrieval. Test both on ProcessBench (GSM8K, MATH, OlympiadBench, OmniMATH).

4

Analyze the difference

Check whether retrieval helps more on the hardest datasets — the same evidence pattern that validated RetrievalPRM.

Techniques & Compute We Need

Techniques

- Sentence-BERT — turns questions into embeddings
- PCA — shrinks and denoises those embeddings
- Cosine similarity — finds the closest matches
- PathFinder-PRM-7B — already-trained, used frozen
- ProcessBench — for testing / scoring

Compute

- No training from scratch — the 7B model is already released on Hugging Face
- Building the retrieval index: cheap, runs on a normal laptop CPU
- Running inference with retrieval-augmented prompts: moderate — a single GPU or API budget for a few thousand calls

Feasibility Checklist



Which component is modified?

The reward-modeling pipeline — adding a retrieval preprocessing step to an existing hierarchical PRM. Clean and well-defined.



Hardware

No training required — PathFinder-PRM's 7B model is already released. Only cheap CPU work (index) + moderate inference (a few thousand calls).



Dataset

Already exists — PathFinder-600K (training data) + ProcessBench (evaluation). No new data collection needed.



Evaluation metric

Already defined — F1 on ProcessBench, directly comparable to numbers both papers already reported.

Passes all four checks cleanly — the lowest compute risk of anything we considered, since we reuse an already-trained model.

Expected Evidence: What Would Prove It's Working

RetrievalPRM's strongest evidence wasn't a single score — it was a pattern: gains grew as problems got harder. We'll look for the same pattern here.

Dataset	OOD severity	What we'd expect
GSM8K	In-distribution	Small or no change vs. baseline
MATH	Moderate OOD	Modest gain from retrieval
OlympiadBench	Severe OOD	Larger gain from retrieval
OmniMATH	Extreme OOD	Largest gain — if the mechanism is real

If gains grow with difficulty, that's causal evidence retrieval is fixing the distribution gap — not just memorizing.

Risks, Limitations & Honest Framing

Strengths

- Clean, answerable research question neither paper tested
- Directly comparable evaluation (F1 on ProcessBench)
- Low compute risk — inference only, no retraining
- Natural continuation of our seminar topic

Open Risks

- Same small retrieval pool limitation as RetrievalPRM
- Retrieval might not help error-typing the way it helped binary grading — a negative result is still a valid, reportable finding
- This is an empirical combination of two existing ideas, not a new invention — we'll frame it that way explicitly

We'll report whatever we find honestly — a clear negative or mixed result, well explained, is still a good project outcome.

Conclusion & Discussion

Summary

- RetrievalPRM showed retrieval helps a binary grader; PathFinder-PRM showed error-typing helps a grader too
- Nobody has combined the two — our project tests whether adding retrieval to an error-typing grader makes it even better, especially on the hardest problems
- No new training, no new data collection — feasible within our compute and time budget

Questions for Discussion

- Does this scope feel right for a project report, or should we narrow / extend it further?
- Is comparing F1 with vs. without retrieval, across all four ProcessBench sub-datasets, sufficient evidence?
- Any concerns about reusing the same retrieval pool as RetrievalPRM?